Data Flow Analysis
Control Flow Graphs

Static testing & Compliers

Modern compilers are useful static analysis tools that can detect defects such as:

- Syntax errors
- Correct data type usage
- Undeclared variables
- Unreachable code
- Variables that are used before they are set. • Conditions that are constant.
- Out-of-bounds addressing
- Function and interface typing

This is assisted by **data flow analysis** and **control flow analysis**.

Data Flow Analysis (1)

Concerned with how data is used on the different paths through the code.

There are 3 different usage states for each data variable.

Undefined (u) The data has no defined value.

Defined (d) The data is assigned a value.

Referenced (r) The data is used.

Data Flow Analysis (2)

Data flow analysis cannot detect errors

But it can identify anomalies which may be high risk features:

ur-anomaly - An undefined data item is read on a program path.

du-anomaly - A data item that has been assigned a value becomes undefined without being used.

dd-anomaly - A data item that has been assigned a value is assigned another value without being used in the meantime

Over to you….

Lets look at an example….

Data Flow Analysis
Example

```
1 class Exchange {
2 public static void main(String[] args) {
3         int Help, Min=2, Max=4;
4         if (Min > Max)
5      {
6             Max = Help;
7             Max = Min;
8                Help = Min;
9         }
10 }
```

Are there any anomalies?

If so what type are they?

Data Flow Analysis
Code Example

```
1 class Exchange {
2 public static void main(String[] args) {
3             int Help, Min=2, Max=4;
4             if (Min > Max)
5         {
6                   Max = Help;
7                   Max = Min;
8                       Help = Min;
9             }
10 }
```

ur-anomaly In line 6 Help is used before it is assigned a value.

dd-anomaly in line 7 Max is assigned a value without using the value it was assigned in lines 6.

du-anomaly in line 8 Help is assigned a value, but is then never used within the scope of Exchange.

Data Flow Analysis

Example

```
1  import java.util.Scanner;
2  class FactorialExample {
3  public static void main(String[] args) {
4  int n, counter, result=1;
5  Scanner in = new Scanner(System.in);
6
7  do{
8           System.out.println("Enter a positive integer: ");
9           n = in.nextInt();
10 } while (n<=0);
11
12 for (counter=1; counter<=n; counter++)
13 {
14          result = result * counter;
15 }
16
17          System.out.println(n + " factorial is: " + result);
18  }
19 }
```

Are there any data flow anomalies?

Path Testing

Control Flow Graphs (CFG)

Program flow graphs

Describes the program control flow. Each branch is shown as a separate path and loops are shown by arrows looping back to the loop condition node

The control flowgraph is a graphical representation of a program’s control structure.

Path testing

- The objective of path testing is to ensure that the set of test cases is such that the desired paths through the program are executed at least once
- The starting point for path testing is a program flow graph that shows nodes representing program decisions and arcs representing the flow of control
- Statements with conditions are therefore nodes in the flow graph

Flowgraphs Consist of Three Primitives

A **decision** is a program point at which the control can diverge.

(e.g., `if` and `case` statements).

A **junction** is a program point where the control flow can merge.

(e.g., `end if`, `end loop`, etc)

A **process block** is a sequence of program statements uninterrupted by either decisions or junctions. (i.e., straight-line code).

A process has one entry and one exit.

A program does not jump into or out of a process.

Paths

A **path** through a program is a sequence of statements that starts at an entry, junction, or decision and ends at another (possibly the same), junction, decision, or exit.

A path may go through several junctions, processes, or decisions, one or more times.

Paths consist of **segments**.

The smallest segment is a link. A **link** is a single process that lies between 2 nodes.

Paths (Cont'd)

The **length** of a path is the number of links in a path.

An **entry/exit path** or a **complete path** is a path that starts at a routine's entry and ends at the same routine's exit.

Paths (Cont'd)

Complete paths are useful for testing because:

- It is difficult to set up and execute paths that start at an arbitrary statement.
- It is difficult to stop at an arbitrary statement without changing the code being tested.
- We think of routines as input/output paths.

Path Selection Criteria

There are many paths between the entry and exit points of a typical routine.

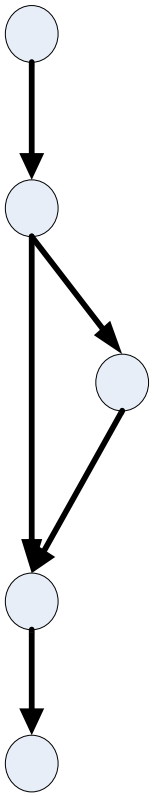
Even a small routine can have a large number of paths.

Over to you…..

Any questions?.....

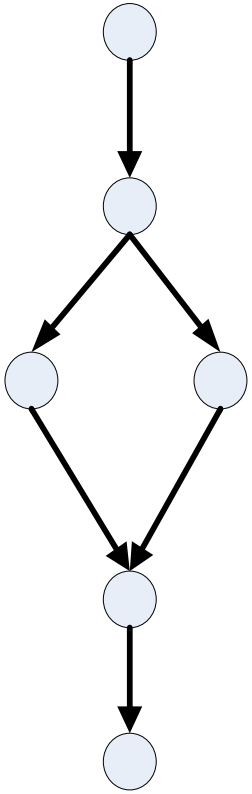
Basic Program Flow Controls

IF Diagram



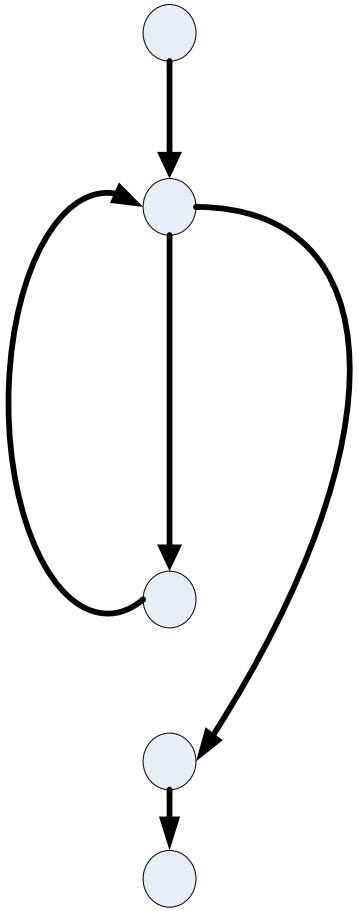
If

IF-THEN-ELSE Diagram



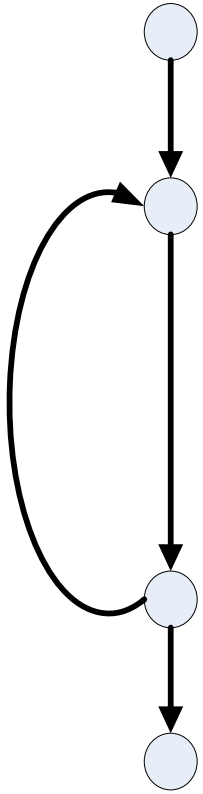
If-then-else

FOR or WHILE Diagram



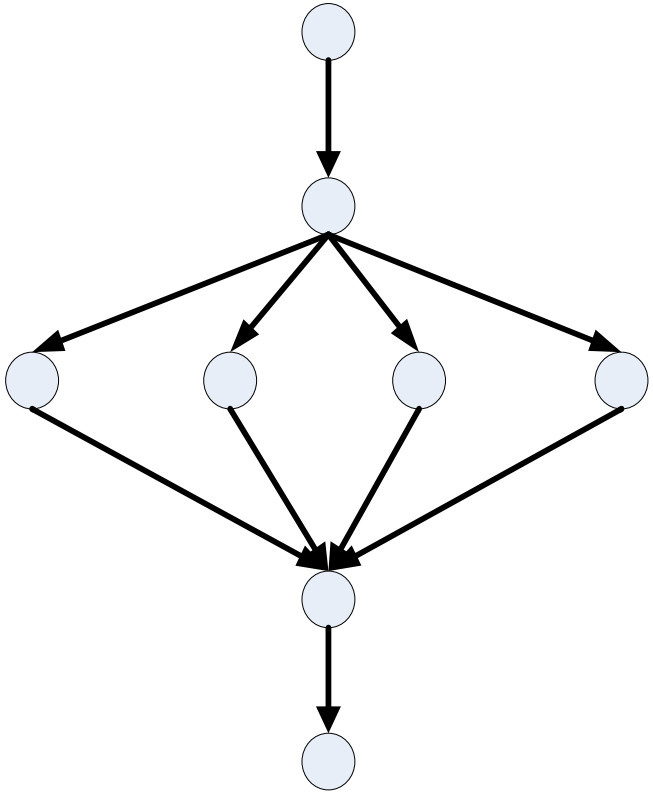
For OR While

DO-WHILE Diagram



Do-while

CASE (switch) Diagram



Case

Over to you…..

An exercise….

Exercise
Can you diagram this code?

```
int process_srb(void)
{
  int srb_count = 0;
  do
  {
      srb_count =
  read_printer(srb_in);
  } while (!srb_count);
    fprintf(logFile,"%s\n",srb_in);
    return(srb_count);
}
```

Step 1
Highlight control flow elements

```
int process_srb(void)
{
  int srb_count = 0;
  do
  {
      srb_count =
  read_printer(srb_in);
  } while (!srb_count);
   fprintf(logFile,"%s\n",srb_in);
   return(srb_count);
}
```

Step 2
Understand what is really going on

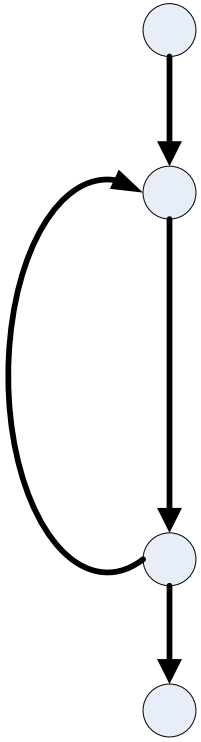
```
int □ □ □ □
{
   □ □ □ □
  do
  {

  } while
    □ □ □ □
    return;
}
```

Did You Get Something Like This?

```
int □ □ □ □
{
      □ □ □ □
     do
     {

     } while
       □ □ □ □
       return;
}
```



Over to you…..

An exercise….

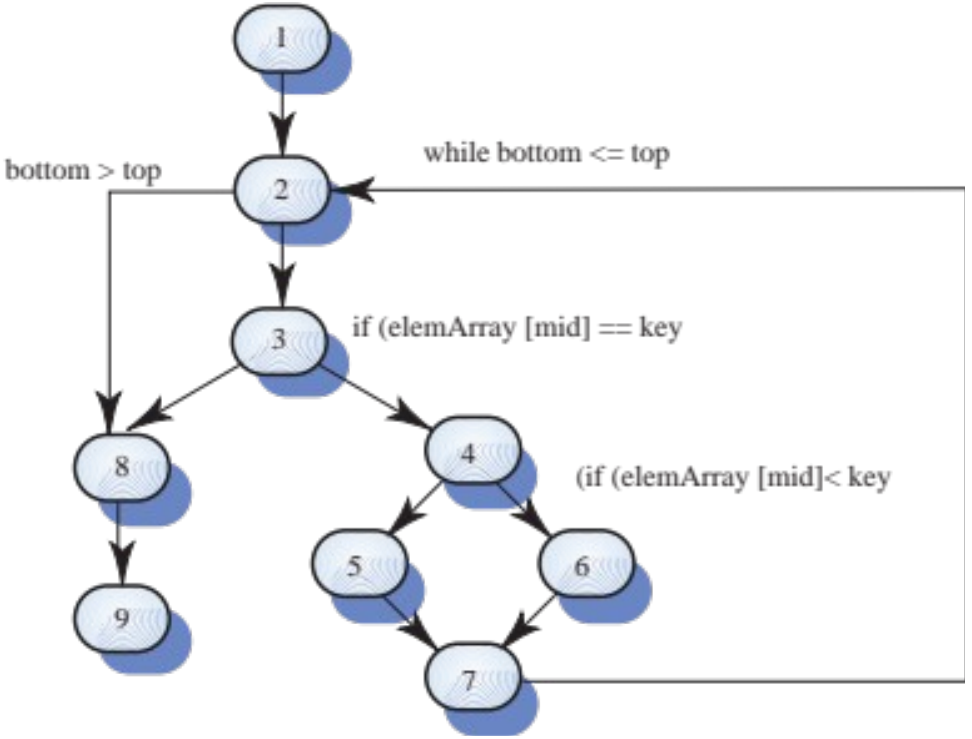
Consider what the diagram for this code would look like….

Binary search (Java)

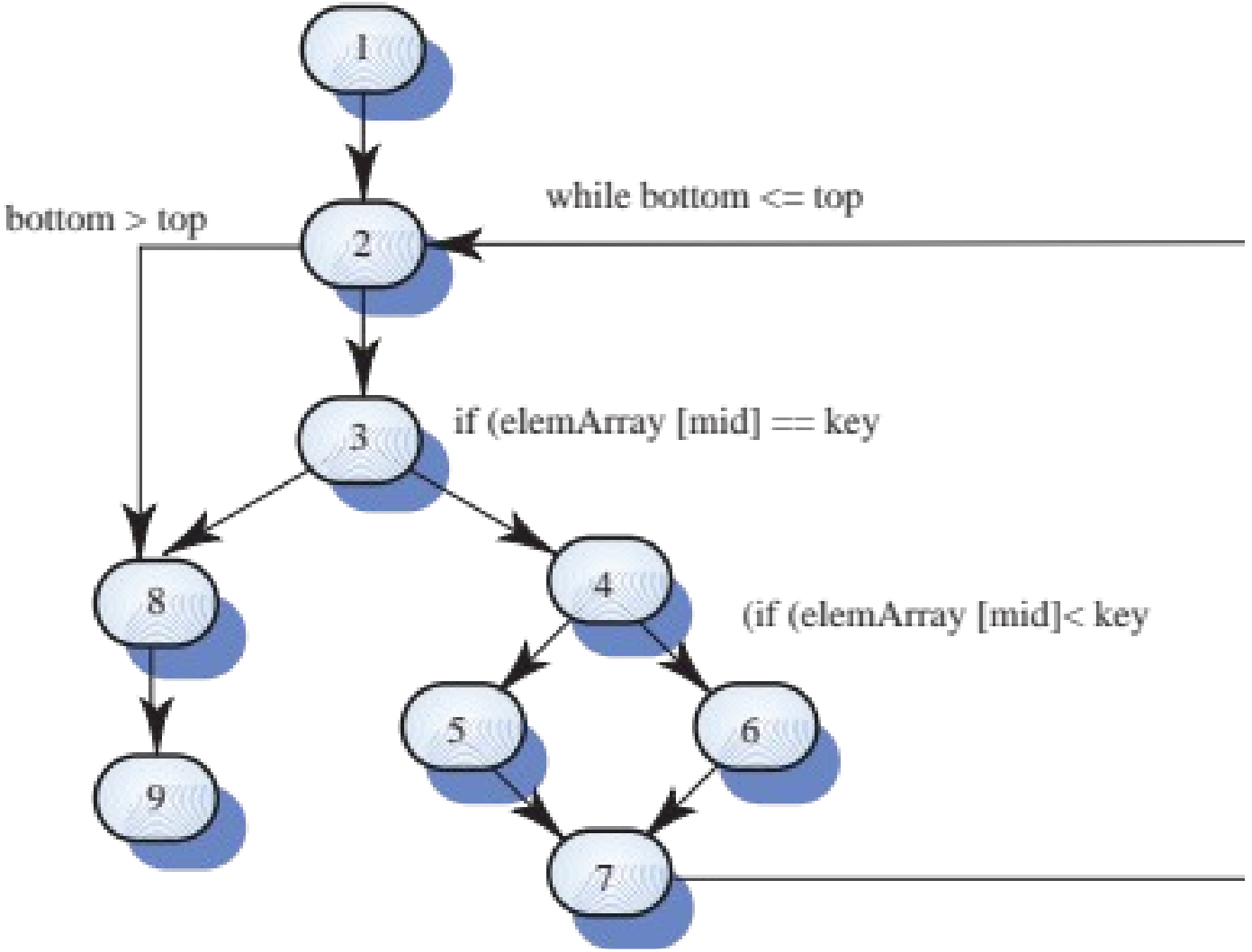
```
class BinSearch {
   public static void search ( int key, int [] elemArray, Result r )
   {
          int bottom = 0 ;
          int top = elemArray.length - 1 ;
          int mid ;
          r.found = false ; r.index = -1 ;
          while ( bottom <= top )
          {
                 mid = (top + bottom) / 2 ;
                 if (elemArray [mid] == key)
                 {
                        r.index = mid ;
                        r.found = true ;
                        return ;
                 } // if part
                 else
                 {
                        if (elemArray [mid] < key)
                               bottom = mid + 1 ;
                        else
                               top = mid - 1 ;
                 }
          } //while loop
   } // search
} //BinSearch
```

Binary search (Java)

```
class BinSearch
{
public static void search XXXXXX
{
   int bottom = 0 ;
   int top = elemArray.length - 1 ;
   int mid ;
   r.found = false ; r.index = -1 ;
   while ( bottom <= top )
   {
      mid = (top + bottom) / 2 ;
      if (elemArray [mid] == key)
      {
         r.index = mid ;
         r.found = true ;
         return ;
      } // if part
      else
      {
         if (elemArray [mid] < key)
            bottom = mid + 1 ;
         else
            top = mid - 1 ;
       }
   } //while loop
 } // search
} //BinSearch
```



Binary search flow graph



Paths and Testing

1, 2, 8, 9

1, 2, 3, 8, 9

1, 2, 3, 4, 6, 7, 2, 8, 9

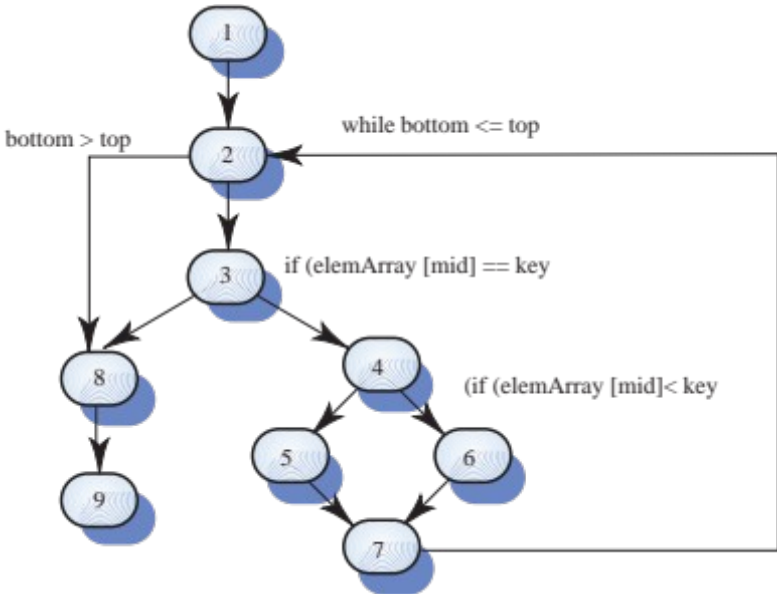
1, 2, 3, 4, 5, 7, 2, 8, 9

1, 2, 3, 4, 6, 7, 2, 3, 8, 9

Etc…

With path testing, test cases should be designed with these paths in mind

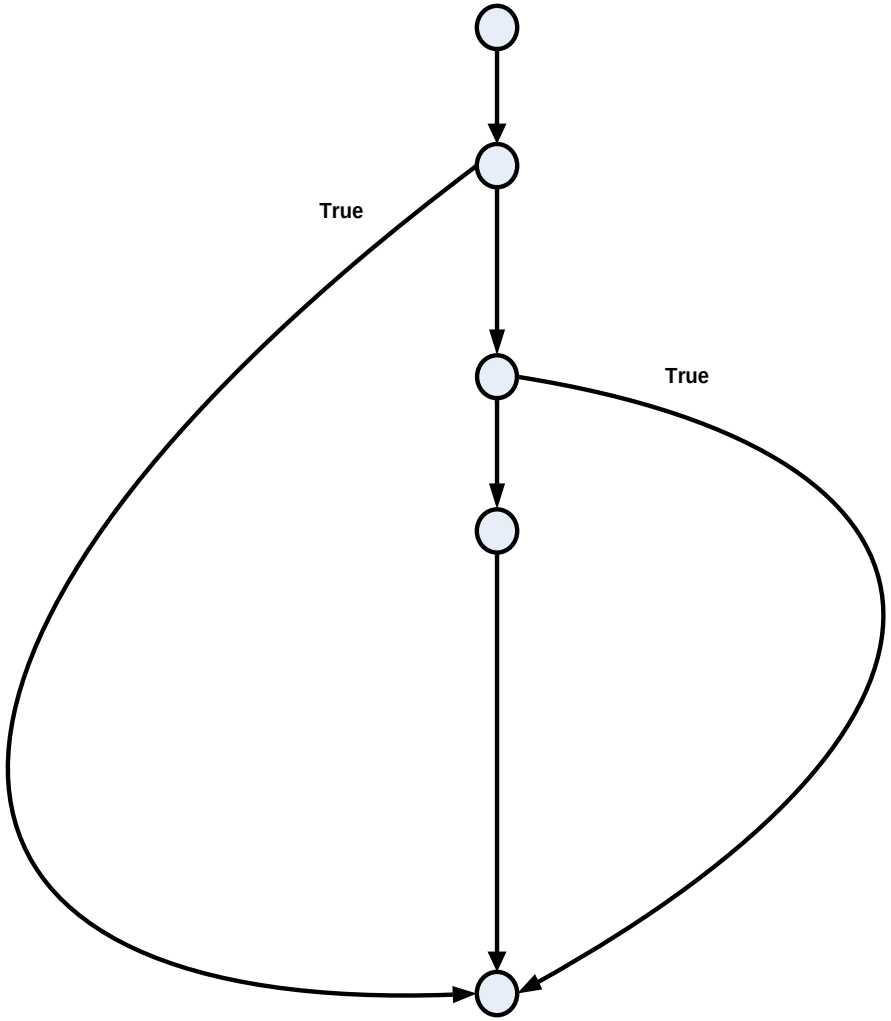
A dynamic program analyser may be used to check that paths have been executed



Exercise
Can you diagram this code?

```
int main (int argc, char *argv[])
{
  signal(SIGINT,catcher);
  if (validate_command_line(argc))
  return(1);
  if (job_initialize(argv))
       return(1);
  processTestList();
  displayResults();
  fprintf(stdout,"Test Complete\n");
  job_termination();
  return(0);
}
```

Did You Get Something Like This?

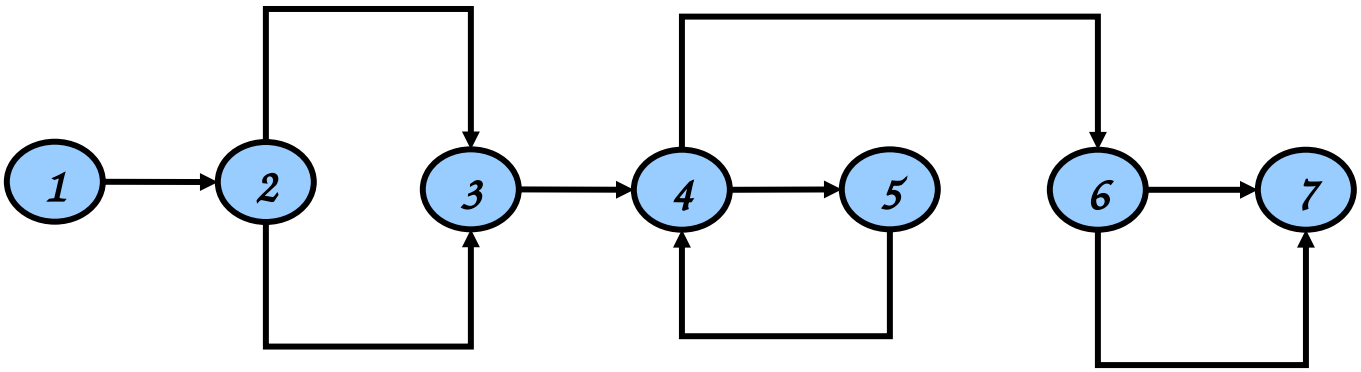


Exponentiation Algorithm

```
1    scanf(“%d %d”,&x, &y);
2    if (y < 0)
          pow = -y;
      else
          pow = y;
3    z = 1.0;
4    while (pow != 0) {
           z = z * x;
           pow = pow - 1;
5           }
6    if (y < 0)
           z = 1.0 / z;
7    printf (“%f”,z);
```

Exponentiation Algorithm

```
1    scanf(“%d %d”,&x, &y);
2    if (y < 0)
          pow = -y;
      else
          pow = y;
3    z = 1.0;
4    while (pow != 0) {
           z = z * x;
           pow = pow - 1;
5           }
6    if (y < 0)
           z = 1.0 / z;
7    printf (“%f”,z);
```

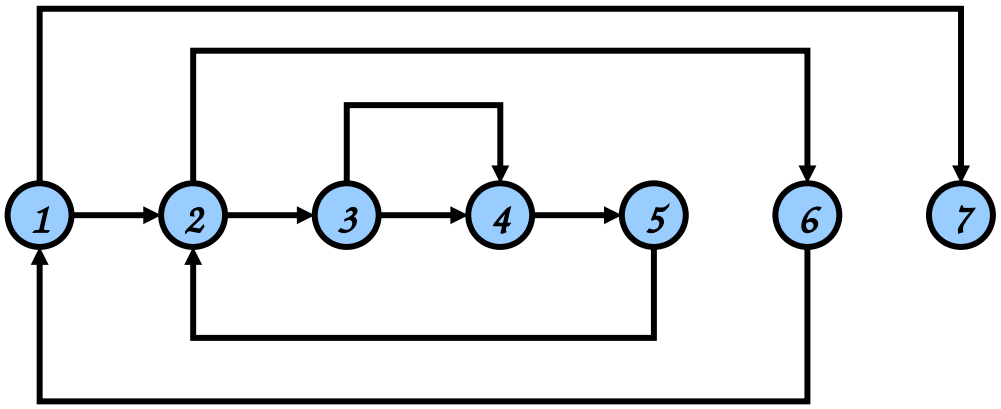


Bubble Sort Algorithm

```
1  for (j=1; j<N; j++)  {
       last = N - j + 1;
2      for (k=1; k<last; k++)  {
3            if (list[k] > list[k+1])  {
                   temp = list[k];
                   list[k] = list[k+1];
                   list[k+1] = temp;
4            }
5      }
6  }
7 print(“Done\n”);
```

Bubble Sort Algorithm

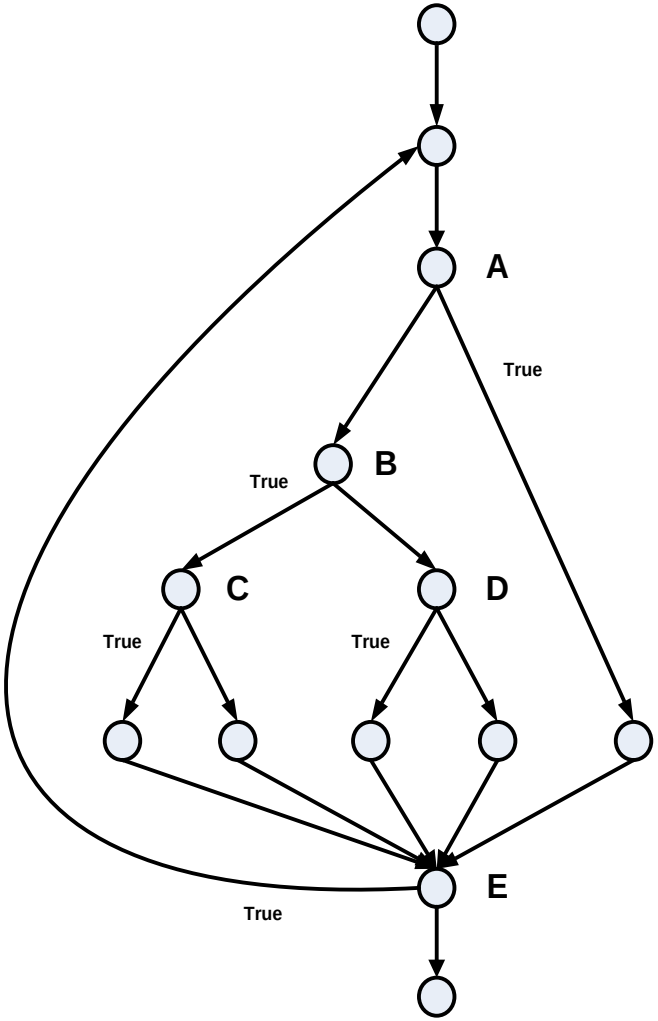
```
1  for (j=1; j<N; j++)  {
       last = N - j + 1;
2      for (k=1; k<last; k++)  {
3            if (list[k] > list[k+1])  {
                   temp = list[k];
                   list[k] = list[k+1];
                   list[k+1] = temp;
4            }
5      }
6  }
7 print("Done\n");
```



Exercise
Can you diagram this code?

```
Do
{
  if (A) then {...};
  else {
    if (B) then {
             if (C) then {...};
             else {…};
    }
    else if (D) then {...};
         else {...};
  }
}
While (E);
```

Example Control Flow Graph



Over to you…..

Any questions?.....